As we are developing SaaS platform, any subscribed user should be able to define their account. Should the user need to define an AES key to encrypt documents and store them in separate folders (linked to the user) and AES key is managed by admin of each user

Mar 15

Architected multi-tenant encryption and storage isolation strategy

Architected multi-tenant encryption and storage isolation strategy

Excellent question! For a proper SaaS multi-tenant platform, we need to add an **Organization/Tenant** model. Here's the architecture:

Multi-Tenant Architecture

```
┌─────────────────────────────────────────────────────────┐
│                        LEXBOX SAAS                      │
├─────────────────────────────────────────────────────────┤
│  Organization A          │  Organization B              │
│  (Law Firm ABC)          │  (Law Firm XYZ)              │
│                          │                              │
│  • Own encryption key    │  • Own encryption key        │
│  • Own document folder   │  • Own document folder       │
│  • Own settings          │  • Own settings              │
│  • Own clients/dossiers  │  • Own clients/dossiers      │
│  • Own users (admin,     │  • Own users (admin,         │
│    lawyers, secretaries) │    lawyers, secretaries)     │
└─────────────────────────────────────────────────────────┘
```

Key Design Decisions

Aspect	Options	Recommendation
Encryption Key	1) Admin enters key manually 2) Auto-generated per org 3) Derived from master key + org ID	**Option 3**: Safest - admin doesn't need to remember, but each org has unique key
Document Storage	1) Single folder with org prefix 2) Separate folders per org	**Option 2**: `/uploads/{org_id}/...` - cleaner isolation
Database	1) Shared DB with `organization_id` 2) Separate DB per org	**Option 1**: Easier to manage, add `organization_id` to all tables

Do you want me to implement this?

It involves:

1. **Organization Model** - name, subscription plan, encryption settings, storage path
2. **Update User Model** - add `organization_id`, `is_org_admin` flag
3. **Update all models** - add `organization_id` to Client, Dossier, Document, Invoice, Settings
4. **Encryption per org** - derive key from `MASTER_KEY + organization_id`
5. **Document storage** - `/uploads/{org_id}/documents/`
6. **Organization Settings** - each org has its own company info, SMTP, billing defaults
7. **Super Admin** - platform-level admin to manage all organizations

---

Questions for you:

1. **Encryption key management**: Should org admins:
   - (A) Enter their own key (they must remember it - if lost, documents unrecoverable)
   - (B) System auto-generates and derives keys (safer, admin doesn't see the key)
   - (C) Admin can view/rotate the key from settings
2. **Super Admin**: Do you need a platform-level "super admin" who can:
   - Create/manage organizations
   - View all orgs' stats (not their data)
   - Manage subscriptions
3. **Subscription plans**: Do you want to add plan limits like:
   - Max users per org
   - Max storage per org
   - Feature restrictions

Let me know your preferences and I'll implement the full multi-tenant architecture!

Do you want me to implement this (multi-tenant architecture)? yes, implement this Also,
We need 1. Encryption key management, I would choose option (B) 1. Platfrom-level Super Admin as you described) 1. Create/manage organizations (so that they request to be included from then we create and manage them). 2. View all orgs' stats (not their data) 3. Manage subscriptions 2. Subscription plans: Do you want to add plan limits, like: * Max users per org * Max storage per org * Feature restrictions

Show more

4:10 PM

Architected phased multi-tenancy rollout with encryption and admin controls

Architected phased multi-tenancy rollout with encryption and admin controls

Let's implement the full multi-tenant architecture. This is a significant change, so I'll do it step by step.

Step 1: Create Organization and Subscription Models

1.1 Create SubscriptionPlan Model

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/SubscriptionPlan.js << 'EOF'
// ====================================================================
// SUBSCRIPTION PLAN MODEL
// ====================================================================
// src/models/SubscriptionPlan.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const SubscriptionPlan = sequelize.define('SubscriptionPlan', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  name: {
    type: DataTypes.STRING(100),
    allowNull: false,
    unique: true
  },
  code: {
    type: DataTypes.STRING(50),
    allowNull: false,
    unique: true
  },
  description: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  price_monthly: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    defaultValue: 0
  },
  price_yearly: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
```

```
    defaultValue: 0
  },
  // Limits
  max_users: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 5
  },
  max_storage_gb: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 10
  },
  max_clients: {
    type: DataTypes.INTEGER,
    allowNull: true // null = unlimited
  },
  max_dossiers: {
    type: DataTypes.INTEGER,
    allowNull: true // null = unlimited
  },
  // Features
  features: {
    type: DataTypes.JSONB,
    allowNull: false,
    defaultValue: {
      document_encryption: true,
      email_invoices: true,
      pdf_export: true,
      api_access: false,
      priority_support: false,
      custom_branding: false,
      audit_logs: false
    }
  },
  is_active: {
    type: DataTypes.BOOLEAN,
    defaultValue: true
  },
  sort_order: {
    type: DataTypes.INTEGER,
    defaultValue: 0
  }
}, {
  tableName: 'subscription_plans',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at'
});

module.exports = SubscriptionPlan;
EOF
```

1.2 Create Organization Model

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Organization.js << 'EOF'
// ====================================================================
// ORGANIZATION MODEL (Multi-Tenant)
// ====================================================================
// src/models/Organization.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');
const crypto = require('crypto');

const Organization = sequelize.define('Organization', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true
  },
  name: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  slug: {
    type: DataTypes.STRING(100),
    allowNull: false,
    unique: true
  },
  // Contact Info
  email: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  phone: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  address: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  website: {
    type: DataTypes.STRING(255),
    allowNull: true
  },
  tax_id: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  // Encryption - auto-generated salt for key derivation
  encryption_salt: {
    type: DataTypes.STRING(64),
    allowNull: false
  },
  // Storage
  storage_path: {
    type: DataTypes.STRING(500),
```

```
    allowNull: false
  },
  storage_used_bytes: {
    type: DataTypes.BIGINT,
    defaultValue: 0
  },
  // Subscription
  subscription_plan_id: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'subscription_plans',
      key: 'id'
    }
  },
  subscription_status: {
    type: DataTypes.ENUM('trial', 'active', 'past_due', 'cancelled',
'suspended'),
    defaultValue: 'trial'
  },
  subscription_started_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  subscription_expires_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  trial_ends_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  billing_cycle: {
    type: DataTypes.ENUM('monthly', 'yearly'),
    defaultValue: 'monthly'
  },
  // Status
  status: {
    type: DataTypes.ENUM('pending', 'active', 'suspended', 'deleted'),
    defaultValue: 'pending'
  },
  // Metadata
  settings: {
    type: DataTypes.JSONB,
    defaultValue: {}
  },
  metadata: {
    type: DataTypes.JSONB,
    defaultValue: {}
  }
}, {
  tableName: 'organizations',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  hooks: {
```

```
    beforeCreate: async (org) => {
      // Generate unique encryption salt
      org.encryption_salt = crypto.randomBytes(32).toString('hex');
      // Generate storage path
      org.storage_path = `organizations/${org.id}`;
      // Generate slug if not provided
      if (!org.slug) {
        org.slug = org.name
          .toLowerCase()
          .replace(/[^a-z0-9]+/g, '-')
          .replace(/^-|-$/g, '') + '-' + Date.now().toString(36);
      }
      // Set trial period (14 days)
      if (!org.trial_ends_at) {
        const trialEnd = new Date();
        trialEnd.setDate(trialEnd.getDate() + 14);
        org.trial_ends_at = trialEnd;
      }
    }
  }
});

module.exports = Organization;
EOF
```

1.3 Create OrganizationRequest Model (for signup requests)

bash

```
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/OrganizationRequest.js << 'EOF'
// ====================================================================
// ORGANIZATION REQUEST MODEL
// ====================================================================
// src/models/OrganizationRequest.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const OrganizationRequest = sequelize.define('OrganizationRequest', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  // Company Info
  organization_name: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  // Contact Person
  contact_first_name: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  contact_last_name: {
```

```
    type: DataTypes.STRING(100),
    allowNull: false
  },
  contact_email: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  contact_phone: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  // Additional Info
  company_size: {
    type: DataTypes.STRING(50),
    allowNull: true // e.g., "1-5", "6-20", "21-50", "50+"
  },
  message: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  // Status
  status: {
    type: DataTypes.ENUM('pending', 'approved', 'rejected'),
    defaultValue: 'pending'
  },
  reviewed_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  },
  reviewed_at: {
    type: DataTypes.DATE,
    allowNull: true
  },
  review_notes: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  // If approved, link to created org
  organization_id: {
    type: DataTypes.UUID,
    allowNull: true,
    references: {
      model: 'organizations',
      key: 'id'
    }
  }
}, {
  tableName: 'organization_requests',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at'
});
```

```
module.exports = OrganizationRequest;
EOF
```

Step 2: Update User Model with Organization and Super Admin

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/User.js <<
'EOF'
// ====================================================================
// USER MODEL (Multi-Tenant)
// ====================================================================
// src/models/User.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');
const bcrypt = require('bcryptjs');

const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  // Organization (null for super admins)
  organization_id: {
    type: DataTypes.UUID,
    allowNull: true,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  email: {
    type: DataTypes.STRING(255),
    allowNull: false,
    validate: {
      isEmail: true
    }
  },
  password: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  first_name: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  last_name: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
```

```
  // Roles: super_admin (platform), org_admin, admin, lawyer, secretary
  role: {
    type: DataTypes.ENUM('super_admin', 'org_admin', 'admin', 'lawyer',
'secretary'),
    defaultValue: 'lawyer'
  },
  // Is this user the organization admin?
  is_org_admin: {
    type: DataTypes.BOOLEAN,
    defaultValue: false
  },
  status: {
    type: DataTypes.ENUM('active', 'inactive', 'suspended'),
    defaultValue: 'active'
  },
  last_login: {
    type: DataTypes.DATE,
    allowNull: true
  },
  phone: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  avatar_url: {
    type: DataTypes.STRING(500),
    allowNull: true
  },
  settings: {
    type: DataTypes.JSONB,
    defaultValue: {}
  }
}, {
  tableName: 'users',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    {
      unique: true,
      fields: ['organization_id', 'email'],
      name: 'users_org_email_unique'
    }
  ],
  hooks: {
    beforeCreate: async (user) => {
      if (user.password) {
        user.password = await bcrypt.hash(user.password, 12);
      }
    },
    beforeUpdate: async (user) => {
      if (user.changed('password')) {
        user.password = await bcrypt.hash(user.password, 12);
      }
    }
  }
```

```
});

// Instance methods
User.prototype.validatePassword = async function(password) {
  return bcrypt.compare(password, this.password);
};

User.prototype.toJSON = function() {
  const values = { ...this.get() };
  delete values.password;
  return values;
};

// Check if user is super admin
User.prototype.isSuperAdmin = function() {
  return this.role === 'super_admin';
};

// Check if user is org admin
User.prototype.isOrgAdmin = function() {
  return this.is_org_admin || this.role === 'org_admin' || this.role ===
'admin';
};

module.exports = User;
EOF
```

Step 3: Update Existing Models with organization_id

3.1 Update Client Model

bash

```
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Client.js
<< 'EOF'
// ====================================================================
// CLIENT MODEL (Multi-Tenant)
// ====================================================================
// src/models/Client.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Client = sequelize.define('Client', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  organization_id: {
    type: DataTypes.UUID,
    allowNull: false,
```

```
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  first_name: {
    type: DataTypes.STRING(100),
    allowNull: false,
    validate: {
      notEmpty: { msg: 'First name is required' },
      len: { args: [2, 100], msg: 'First name must be between 2 and 100
characters' }
    }
  },
  last_name: {
    type: DataTypes.STRING(100),
    allowNull: false,
    validate: {
      notEmpty: { msg: 'Last name is required' },
      len: { args: [2, 100], msg: 'Last name must be between 2 and 100
characters' }
    }
  },
  email: {
    type: DataTypes.STRING(255),
    allowNull: true,
    validate: {
      isEmail: { msg: 'Please provide a valid email address' }
    }
  },
  phone: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  address: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  personal_number: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  date_of_birth: {
    type: DataTypes.DATEONLY,
    allowNull: true
  },
  nationality: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  client_type: {
    type: DataTypes.ENUM('individual', 'company'),
    defaultValue: 'individual'
  },
  company_name: {
    type: DataTypes.STRING(255),
    allowNull: true
```

```
  },
  company_registration_number: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  status: {
    type: DataTypes.ENUM('active', 'inactive', 'archived'),
    defaultValue: 'active'
  },
  registration_date: {
    type: DataTypes.DATE,
    defaultValue: DataTypes.NOW
  },
  notes: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  created_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  },
  updated_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  }
}, {
  tableName: 'clients',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { fields: ['organization_id'] },
    { fields: ['organization_id', 'email'] },
    { fields: ['organization_id', 'status'] }
  ]
});

module.exports = Client;
EOF
```

3.2 Update Dossier Model

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Dossier.js
<< 'EOF'
// ====================================================================
```

```
// DOSSIER MODEL (Multi-Tenant)
// ====================================================================
// src/models/Dossier.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Dossier = sequelize.define('Dossier', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  organization_id: {
    type: DataTypes.UUID,
    allowNull: false,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  client_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: 'clients',
      key: 'id'
    }
  },
  dossier_number: {
    type: DataTypes.STRING(50),
    allowNull: false
  },
  title: {
    type: DataTypes.STRING(255),
    allowNull: true
  },
  description: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  legal_issue_type: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  legal_issue_subtype: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  status: {
    type: DataTypes.ENUM('active', 'pending', 'closed', 'archived'),
    defaultValue: 'active'
  },
  priority: {
    type: DataTypes.ENUM('low', 'medium', 'high', 'urgent'),
    defaultValue: 'medium'
```

```
  },
  assigned_to: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  },
  opened_date: {
    type: DataTypes.DATEONLY,
    defaultValue: DataTypes.NOW
  },
  closed_date: {
    type: DataTypes.DATEONLY,
    allowNull: true
  },
  total_billed: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
  total_paid: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
  metadata: {
    type: DataTypes.JSONB,
    defaultValue: {}
  },
  created_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  }
}, {
  tableName: 'dossiers',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { fields: ['organization_id'] },
    { unique: true, fields: ['organization_id', 'dossier_number'] },
    { fields: ['organization_id', 'status'] },
    { fields: ['organization_id', 'client_id'] }
  ]
});

module.exports = Dossier;
EOF
```

3.3 Update Document Model

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Document.js << 'EOF'
// ====================================================================
// DOCUMENT MODEL (Multi-Tenant)
// ====================================================================
// src/models/Document.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Document = sequelize.define('Document', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  organization_id: {
    type: DataTypes.UUID,
    allowNull: false,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  dossier_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: 'dossiers',
      key: 'id'
    }
  },
  filename: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  original_filename: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  file_path: {
    type: DataTypes.STRING(500),
    allowNull: false
  },
  file_size: {
    type: DataTypes.INTEGER,
    allowNull: false
  },
  mime_type: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  category: {
    type: DataTypes.STRING(50),
    allowNull: true,
```

```
    defaultValue: 'other'
  },
  physical_location: {
    type: DataTypes.STRING(100),
    allowNull: true
  },
  description: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  document_date: {
    type: DataTypes.DATEONLY,
    allowNull: true
  },
  is_confidential: {
    type: DataTypes.BOOLEAN,
    defaultValue: false
  },
  metadata: {
    type: DataTypes.JSONB,
    defaultValue: {}
  },
  uploaded_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  }
}, {
  tableName: 'documents',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { fields: ['organization_id'] },
    { fields: ['organization_id', 'dossier_id'] }
  ]
});

module.exports = Document;
EOF
```

3.4 Update TimelineNode Model

bash

```
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/TimelineNode.js << 'EOF'
// ====================================================================
// TIMELINE NODE MODEL (Multi-Tenant)
// ====================================================================
// src/models/TimelineNode.js
```

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const TimelineNode = sequelize.define('TimelineNode', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  organization_id: {
    type: DataTypes.UUID,
    allowNull: false,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  dossier_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: 'dossiers',
      key: 'id'
    }
  },
  node_type: {
    type: DataTypes.ENUM('registration', 'classification', 'activity',
'document', 'milestone', 'billing'),
    allowNull: false,
    defaultValue: 'activity'
  },
  activity_type: {
    type: DataTypes.STRING(50),
    allowNull: true
  },
  title: {
    type: DataTypes.STRING(255),
    allowNull: false
  },
  description: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  activity_date: {
    type: DataTypes.DATE,
    allowNull: false,
    defaultValue: DataTypes.NOW
  },
  hours_worked: {
    type: DataTypes.DECIMAL(5, 2),
    defaultValue: 0.00
  },
  hourly_rate: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
```

```
  billing_amount: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
  is_billable: {
    type: DataTypes.BOOLEAN,
    defaultValue: false
  },
  is_billed: {
    type: DataTypes.BOOLEAN,
    defaultValue: false
  },
  invoice_id: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'invoices',
      key: 'id'
    }
  },
  document_id: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'documents',
      key: 'id'
    }
  },
  status: {
    type: DataTypes.ENUM('pending', 'in_progress', 'completed', 'cancelled'),
    defaultValue: 'completed'
  },
  priority: {
    type: DataTypes.ENUM('low', 'medium', 'high', 'urgent'),
    defaultValue: 'medium'
  },
  metadata: {
    type: DataTypes.JSONB,
    defaultValue: {}
  },
  created_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  },
  updated_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  }
}, {
```

```
  tableName: 'timeline_nodes',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { fields: ['organization_id'] },
    { fields: ['organization_id', 'dossier_id'] }
  ]
});

module.exports = TimelineNode;
EOF
```

3.5 Update Invoice Model

bash

```
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Invoice.js
<< 'EOF'
// ====================================================================
// INVOICE MODEL (Multi-Tenant)
// ====================================================================
// src/models/Invoice.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Invoice = sequelize.define('Invoice', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  organization_id: {
    type: DataTypes.UUID,
    allowNull: false,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  dossier_id: {
    type: DataTypes.INTEGER,
    allowNull: false,
    references: {
      model: 'dossiers',
      key: 'id'
    }
  },
  invoice_number: {
    type: DataTypes.STRING(50),
    allowNull: false
  },
  status: {
```

```
    type: DataTypes.ENUM('draft', 'sent', 'partial', 'paid', 'overdue',
'cancelled'),
    defaultValue: 'draft'
  },
  issue_date: {
    type: DataTypes.DATEONLY,
    allowNull: false,
    defaultValue: DataTypes.NOW
  },
  due_date: {
    type: DataTypes.DATEONLY,
    allowNull: false
  },
  subtotal: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    defaultValue: 0.00
  },
  tax_rate: {
    type: DataTypes.DECIMAL(5, 2),
    defaultValue: 0.00
  },
  tax_amount: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
  total_amount: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    defaultValue: 0.00
  },
  amount_paid: {
    type: DataTypes.DECIMAL(10, 2),
    defaultValue: 0.00
  },
  payment_terms: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  notes: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  metadata: {
    type: DataTypes.JSONB,
    defaultValue: {}
  },
  created_by: {
    type: DataTypes.INTEGER,
    allowNull: true,
    references: {
      model: 'users',
      key: 'id'
    }
  }
}, {
  tableName: 'invoices',
```

```
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { fields: ['organization_id'] },
    { unique: true, fields: ['organization_id', 'invoice_number'] },
    { fields: ['organization_id', 'dossier_id'] }
  ]
});

module.exports = Invoice;
EOF
```

3.6 Update Settings Model for multi-tenant

bash

```
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Settings.js << 'EOF'
// ====================================================================
// SETTINGS MODEL (Multi-Tenant)
// ====================================================================
// src/models/Settings.js

const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Settings = sequelize.define('Settings', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true
  },
  // null = global/platform settings, UUID = organization settings
  organization_id: {
    type: DataTypes.UUID,
    allowNull: true,
    references: {
      model: 'organizations',
      key: 'id'
    }
  },
  key: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  value: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  type: {
    type: DataTypes.ENUM('string', 'number', 'boolean', 'json'),
    defaultValue: 'string'
  },
  category: {
```

```
    type: DataTypes.STRING(50),
    allowNull: false,
    defaultValue: 'general'
  },
  label: {
    type: DataTypes.STRING(255),
    allowNull: true
  },
  description: {
    type: DataTypes.TEXT,
    allowNull: true
  },
  is_sensitive: {
    type: DataTypes.BOOLEAN,
    defaultValue: false
  }
}, {
  tableName: 'settings',
  underscored: true,
  timestamps: true,
  createdAt: 'created_at',
  updatedAt: 'updated_at',
  indexes: [
    { unique: true, fields: ['organization_id', 'key'] }
  ]
});

module.exports = Settings;
EOF
```

Step 4: Update models/index.js with all associations

bash

```bash
cat > /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/index.js
<< 'EOF'
// ====================================================================
// MODEL ASSOCIATIONS (Multi-Tenant)
// ====================================================================
// src/models/index.js

const User = require('./User');
const Client = require('./Client');
const Dossier = require('./Dossier');
const TimelineNode = require('./TimelineNode');
const Document = require('./Document');
const Invoice = require('./Invoice');
const InvoiceLineItem = require('./InvoiceLineItem');
const Payment = require('./Payment');
const Settings = require('./Settings');
const Organization = require('./Organization');
const SubscriptionPlan = require('./SubscriptionPlan');
const OrganizationRequest = require('./OrganizationRequest');
```

```
// ======================================================================
// ORGANIZATION ASSOCIATIONS
// ======================================================================

// Organization <-> SubscriptionPlan
Organization.belongsTo(SubscriptionPlan, {
  foreignKey: 'subscription_plan_id',
  as: 'subscriptionPlan'
});
SubscriptionPlan.hasMany(Organization, {
  foreignKey: 'subscription_plan_id',
  as: 'organizations'
});

// Organization <-> Users
Organization.hasMany(User, {
  foreignKey: 'organization_id',
  as: 'users'
});
User.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// Organization <-> Clients
Organization.hasMany(Client, {
  foreignKey: 'organization_id',
  as: 'clients'
});
Client.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// Organization <-> Dossiers
Organization.hasMany(Dossier, {
  foreignKey: 'organization_id',
  as: 'dossiers'
});
Dossier.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// Organization <-> Documents
Organization.hasMany(Document, {
  foreignKey: 'organization_id',
  as: 'documents'
});
Document.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});
```

```
// Organization <-> TimelineNodes
Organization.hasMany(TimelineNode, {
  foreignKey: 'organization_id',
  as: 'timelineNodes'
});
TimelineNode.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// Organization <-> Invoices
Organization.hasMany(Invoice, {
  foreignKey: 'organization_id',
  as: 'invoices'
});
Invoice.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// Organization <-> Settings
Organization.hasMany(Settings, {
  foreignKey: 'organization_id',
  as: 'settings'
});
Settings.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// OrganizationRequest <-> Organization
OrganizationRequest.belongsTo(Organization, {
  foreignKey: 'organization_id',
  as: 'organization'
});

// ====================================================================
// CLIENT ASSOCIATIONS
// ====================================================================

Client.hasMany(Dossier, {
  foreignKey: 'client_id',
  as: 'dossiers'
});
Dossier.belongsTo(Client, {
  foreignKey: 'client_id',
  as: 'client'
});

// ====================================================================
// DOSSIER ASSOCIATIONS
// ====================================================================

Dossier.hasMany(TimelineNode, {
  foreignKey: 'dossier_id',
```

```
  as: 'timelineNodes'
});
TimelineNode.belongsTo(Dossier, {
  foreignKey: 'dossier_id',
  as: 'dossier'
});

Dossier.hasMany(Document, {
  foreignKey: 'dossier_id',
  as: 'documents'
});
Document.belongsTo(Dossier, {
  foreignKey: 'dossier_id',
  as: 'dossier'
});

Dossier.hasMany(Invoice, {
  foreignKey: 'dossier_id',
  as: 'invoices'
});
Invoice.belongsTo(Dossier, {
  foreignKey: 'dossier_id',
  as: 'dossier'
});

// ====================================================================
// DOCUMENT & TIMELINE ASSOCIATIONS
// ====================================================================

Document.hasMany(TimelineNode, {
  foreignKey: 'document_id',
  as: 'timelineNodes'
});
TimelineNode.belongsTo(Document, {
  foreignKey: 'document_id',
  as: 'document'
});

// ====================================================================
// INVOICE ASSOCIATIONS
// ====================================================================

Invoice.hasMany(InvoiceLineItem, {
  foreignKey: 'invoice_id',
  as: 'lineItems',
  onDelete: 'CASCADE'
});
InvoiceLineItem.belongsTo(Invoice, {
  foreignKey: 'invoice_id',
  as: 'invoice'
});

Invoice.hasMany(Payment, {
  foreignKey: 'invoice_id',
  as: 'payments',
  onDelete: 'CASCADE'
```

```
});
Payment.belongsTo(Invoice, {
  foreignKey: 'invoice_id',
  as: 'invoice'
});

TimelineNode.hasOne(InvoiceLineItem, {
  foreignKey: 'timeline_node_id',
  as: 'invoiceLineItem'
});
InvoiceLineItem.belongsTo(TimelineNode, {
  foreignKey: 'timeline_node_id',
  as: 'timelineNode'
});

// ======================================================================
// USER ASSOCIATIONS
// ======================================================================

User.hasMany(Document, {
  foreignKey: 'uploaded_by',
  as: 'uploaded_documents'
});
Document.belongsTo(User, {
  foreignKey: 'uploaded_by',
  as: 'uploader'
});

User.hasMany(Dossier, {
  foreignKey: 'assigned_to',
  as: 'assigned_dossiers'
});
Dossier.belongsTo(User, {
  foreignKey: 'assigned_to',
  as: 'assignedLawyer'
});

User.hasMany(Client, {
  foreignKey: 'created_by',
  as: 'created_clients'
});
Client.belongsTo(User, {
  foreignKey: 'created_by',
  as: 'creator'
});

User.hasMany(Client, {
  foreignKey: 'updated_by',
  as: 'updated_clients'
});
Client.belongsTo(User, {
  foreignKey: 'updated_by',
  as: 'updater'
});

User.hasMany(Dossier, {
```

```
  foreignKey: 'created_by',
  as: 'created_dossiers'
});
Dossier.belongsTo(User, {
  foreignKey: 'created_by',
  as: 'dossierCreator'
});

User.hasMany(TimelineNode, {
  foreignKey: 'created_by',
  as: 'created_timeline_nodes'
});
TimelineNode.belongsTo(User, {
  foreignKey: 'created_by',
  as: 'creator'
});

User.hasMany(TimelineNode, {
  foreignKey: 'updated_by',
  as: 'updated_timeline_nodes'
});
TimelineNode.belongsTo(User, {
  foreignKey: 'updated_by',
  as: 'updater'
});

User.hasMany(Invoice, {
  foreignKey: 'created_by',
  as: 'created_invoices'
});
Invoice.belongsTo(User, {
  foreignKey: 'created_by',
  as: 'creator'
});

User.hasMany(Payment, {
  foreignKey: 'recorded_by',
  as: 'recorded_payments'
});
Payment.belongsTo(User, {
  foreignKey: 'recorded_by',
  as: 'recorder'
});

module.exports = {
  User,
  Client,
  Dossier,
  TimelineNode,
  Document,
  Invoice,
  InvoiceLineItem,
  Payment,
  Settings,
  Organization,
  SubscriptionPlan,
```

```
  OrganizationRequest
};
EOF
```

This is a lot of changes. Let me know when you've saved these files and I'll continue with:

1. **Organization Encryption Utility** - derive keys per org
2. **Tenant Middleware** - auto-filter by organization
3. **Super Admin Service & Controller** - manage orgs
4. **Update all services** to use organization_id
5. **Database seeding** - create default plans and super admin

Ready to continue?

how came that code e.g., for TimelineNode.js is shorter that the existing one I have currently running? Basically all clasess ju provided are shorter

4:35 PM

Recognized oversight in replacing models instead of augmenting them

Recognized oversight in replacing models instead of augmenting them

You're absolutely right! I apologize - I was providing simplified versions that might be missing fields you already have.

The correct approach is to ADD the `organization_id` field to your existing models, not replace them entirely.

Let me do this properly. Show me your current TimelineNode.js and I'll add only what's needed:

bash

```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/TimelineNode.js
```

For each model, we only need to ADD:

1. The `organization_id` field
2. An index on `organization_id`

For example, to your existing TimelineNode.js, we just add:

javascript

```
// ADD this field after 'id':
organization_id: {
  type: DataTypes.UUID,
  allowNull: false,
  references: {
    model: 'organizations',
    key: 'id'
  }
},
```

```
// ADD this to indexes array:
{ fields: ['organization_id'] },
{ fields: ['organization_id', 'dossier_id'] }
```

Let me see your existing models so I can provide the correct additions:

bash
```
ls -la /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/
```

bash
```
wc -l /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/*.js
```

Share the outputs and I'll provide **patch-style additions** instead of full replacements!

I reverted changes in the existing files. Please add me exactly what to add instead of changing entire file.

For the new files I have created them.

4:42 PM

Identified incremental field additions across existing models

Identified incremental field additions across existing models

Let me see your current models to provide exact additions:

bash
```
head -50 /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/User.js
```

bash
```
head -40 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/Client.js
```

bash
```
head -40 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/Dossier.js
```

bash
```
head -30 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/Document.js
```

bash
```
head -30 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/TimelineNode.js
```

bash
```
head -30 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/Invoice.js
```

bash
```
head -30 /Users/imeri/Desktop/Dev/LexBoxDev/backend-
lexbox/src/models/Settings.js
```

Share the outputs and I'll provide exact line-by-line additions for each file!

head -50 /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/User.js const { DataTypes } = require('sequelize'); const

bcrypt = require('bcrypt'); const sequelize = require('../config/database'); const User = sequelize.define('User', { id: { type: DataTypes.INTEGER, primaryKey: true, autoIncrement: true }, uuid: { type: DataTypes.UUID, defaultValue: DataTypes.UUIDV4, unique: true, allowNull: false }, username: { type: DataTypes.STRING(50), unique: true, allowNull: false }, email: { type: DataTypes.STRING(255), unique: true, allowNull: false, validate: { isEmail: true } }, password_hash: { type: DataTypes.STRING(255), allowNull: false }, first_name: { type: DataTypes.STRING(100), allowNull: false }, last_name: { type: DataTypes.STRING(100), allowNull: false }, role: { type: DataTypes.ENUM('admin', 'lawyer', 'secretary'), allowNull: false, defaultValue: 'lawyer' }, is_active: { type: DataTypes.BOOLEAN, defaultValue: true },

Show more

4:43 PM

Analyzed model structure and identified necessary field additions

Analyzed model structure and identified necessary field additions

Good! Now let me see the rest of each file to give precise additions:

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/User.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Client.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Dossier.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Document.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/TimelineNode.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Invoice.js
```

bash
```
cat /Users/imeri/Desktop/Dev/LexBoxDev/backend-lexbox/src/models/Settings.js
```

Please share all 7 outputs so I can give you exact line additions!